
Concurrencia y Paralelismo

Bloque I: Concurrencia

Junio 2024

1. Threads que esperan por otros [2 puntos]

Queremos implementar usando mutexes y variables de condición las siguientes funcionalidades:

- a) Todos los threads trabajadores tienen que esperar a que el thread principal llegue a un punto determinado para continuar su ejecución (`barrier_worker()`). El thread principal indica a los threads que ha llegado al punto determinado llamando a la función `barrier_main()`.

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    ...
};

void barrier_worker(struct barrier *b);
void barrier_main(struct barrier *b);
```

- b) Todos los threads tienen que esperar a que el resto de threads lleguen al punto de sincronización. El punto de sincronización se marca con la llamada a `barrier_sync()`. Cuando el último thread llega al punto de sincronización todos continúan.

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    ...
};

void barrier_sync(struct barrier *b);
```

```
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_trylock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Solución:

a)

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    int reached; // Initialized to 0
};

void barrier_worker(struct barrier *b) {
    pthread_mutex_lock(&b->m);
    if(!b->reached)
        pthread_cond_wait(&b->w, &b->m);
    pthread_mutex_unlock(&b->m);
}

void barrier_main(struct barrier *b) {
    pthread_mutex_lock(&b->m);
    b->reached = 1;
    pthread_cond_broadcast(&b->w);
    pthread_mutex_unlock(&b->m);
}
```

b)

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    int n_threads;
};

void barrier_sync(struct barrier *b) {
    pthread_mutex_lock(&b->m);
    b->n_threads--;
    if(b->n_threads > 0)
        pthread_cond_wait(&b->w, &b->m);
    else
        pthread_cond_broadcast(&b->w);
    pthread_mutex_unlock(&b->m);
}
```

2. Secuencia de operaciones [1.5 puntos]

Un array de datos `d` puede ser accedido por varios threads simultaneamente para hacer una operación `do_op` sobre una de sus posiciones. Esta operación tiene efectos secundarios (no es pura). Para proteger el acceso a esos datos existe un array de mutex `d_m`, donde cada posición de `d_m` protege uno de los datos del array.

Vamos a implementar una operación `do_op_seq` que realice una secuencia de operaciones `do_op` especificada por un array que indica, en orden, las posiciones sobre las que hay que operar. Por ejemplo, para hacer la operación sobre las posiciones 1, 5 y 3:

```
int ids[] = {1, 5, 3};
do_op_seq(ids, 3);
```

La secuencia de operaciones no debe realizarse hasta que se hayan bloqueado todas las posiciones afectadas, y debe respetarse el orden indicado por el array de operaciones.

```
void do_op(data d);
void sort(int arr[], int arr_size); // Sort function for int arrays

data d[N];
pthread_mutex_t d_m[N];

void do_op_seq(int ids[], int ids_len) {
    for(int i=0; i < ids_len; i++)
        do_op(d[ids[i]]);
}
```

Implemente la protección de la secuencia de operaciones en `do_op_seq`.

Solución:

```
void do_op(data d);
void sort(int arr[], int arr_size);

data d[N];
pthread_mutex_t d_m[N];

void do_op_seq(int ids[], int ids_len) {
    int sorted_ids[ids_len];

    for(int i=0; i < ids_len; i++)
        sorted_ids[i] = ids[i];

    sort(sorted_ids, ids_len);

    for(int i=0; i < ids_len; i++)
        pthread_mutex_lock(d_m[sorted_ids[i]]);

    for(int i=0; i < ids_len; i++)
        do_op(d[ids[i]]);

    for(int i=0; i < ids_len; i++)
        pthread_mutex_unlock(d_m[sorted_ids[i]]);
}
```

3. Servidor de datos [1.5 puntos]

Escriba un módulo que permita crear procesos servidor con la siguiente interfaz:

- `start()`, que arranca un proceso servidor y devuelve su PID.
- `put(S, Data)`, donde `S` es el PID de un proceso servidor, y `Data` un dato arbitrario. El dato debe quedar almacenado en el servidor.
- `get(S, Proc)`, donde `S` es el PID de un proceso servidor, y `Proc` el PID de un proceso cualquiera. La función debe devolver una lista con todos los datos almacenados por `Proc`.

```
-module(store).  
  
-export([start/0, get/2, put/2]).  
  
start() ->  
    spawn(?MODULE, init, []).  
  
put(S, Data) ->  
    ...  
  
get(S, Proc) ->  
    ...  
  
init() ->  
    ...
```

Solución:

```
-module(store).  
  
-export([start/0, get/2, put/2]).  
  
start() ->  
    spawn(?MODULE, init, []).  
  
put(S, Data) ->  
    S ! {put, Data, self()}.  
  
get(S, Proc) ->  
    S ! {get, Proc, self()},  
    receive  
        {get_reply, L} -> L  
    end.  
  
init() ->  
    loop([]).  
  
loop(L) ->  
    receive  
        {get, Proc, From} ->  
            From ! {get_reply, [Data || {Data, Owner} <- L, Owner == Proc]},  
            loop(L);  
        {put, Data, From} ->  
            loop([Data, From] | L)  
    end.
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria ordinaria

30 de mayo de 2024

AVISO: No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregarán por separado.

- [2.5p] 1. **Conceptos de paralelismo.** La escucha de maquetas de jóvenes promesas del reguetón es la actividad que provoca más bajas laborales en una discográfica, por lo que se desea automatizarla usando IA. Las maquetas son ficheros de audio de entre 3 y 6 minutos y el algoritmo de análisis puntúa cada una en varios aspectos, requiriendo 8 segundos de computación por cada segundo de audio, y no pudiéndose paralelizar el análisis de una maqueta individual. Tras obtener las puntuaciones de las n maquetas de cada día, el programa hace un postprocesado que incluye el ordenamiento de las mismas de acuerdo a sus puntuaciones y el envío de los resultados a los interesados, para lo cual se usa un algoritmo no paralelizable que requiere $n \times \log_2(n)$ segundos.

Cada día hay que evaluar 64 maquetas, por lo que se desea paralelizar el proceso entre varios servidores. El tiempo medio de lectura de disco de cada canción son 0.1 segundos, mientras que el tiempo de transmisión tanto de las maquetas como de las puntuaciones es despreciable, y sólo uno de los servidores contiene el disco con las maquetas. Además, al leer o recibir una maqueta se sabe inmediatamente su duración.

Responde RAZONADAMENTE a las siguientes cuestiones:

- (a) [0.5p] Determina qué tipo de descomposición y asignación de tareas aplicarías.
- (b) [0.75p] Calcula, explicando los pasos y parámetros que usas, la máxima aceleración teórica que se podría obtener.
- (c) [0.75p] Calcula para el caso anterior, explicando los pasos y parámetros que usas, y suponiendo que se usó el menor número posible de servidores para obtener esa aceleración:
 - Eficiencia paralela.
 - Coste
 - Sobrecarga
- (d) [0.5p] Supongamos que el algoritmo de IA se cambiase para poder ser capaz a menudo, pero no siempre y no siendo posible saberlo de antemano, de puntuar una maqueta tras sólo procesar un tercio de la misma. ¿Cambiarías la descomposición y/o asignación de tareas? De ser así, ¿cómo?

- [2.5p] 2. **Diseño de algoritmos paralelos.** Los siguientes algoritmos implementan una función que calcula la desviación típica (σ) de los valores de un vector ' v ' de longitud ' N '. El resultado es idéntico en ambas funciones:

$$\sigma = \sqrt{\frac{1}{N} \sum_i (v_i - \mu)^2}$$

Algoritmo A

```
double stdev_v1(double *v, int N) {
    double sum, sd, media;

    sum = 0;
    for (i=0; i<N; i++)
        sum += v[i]

    media = sum / N;

    sum = 0;
    for (i=0; i<N; i++)
        sum += pow(v[i] - media, 2);

    sd = sqrt(sum / N);

    return sd;
}
```

Algoritmo B

```
double stdev_v2(double *v, int N) {
    double sum, sumsq, sd, media;

    sum = sumsq = 0
    for (i=0; i<N; i++) {
        sum += v[i]
        sumsq += pow(v[i], 2);
    }

    media = sum / N;

    sd = sqrt((sumsq - 2*media*sum +
                N*pow(media,2) / N));

    return sd;
}
```

Queremos convertir la función del **Algoritmo A** en una operación colectiva MPI que tendrá la siguiente firma: “`double stdev_mpi(double *v, int N, int R, int P, int root)`”.

R es el rango del proceso, P es el número de procesos que se ejecutan en paralelo y root es el rango del proceso raíz. Se asume lo siguiente:

- El vector ' v ' sólo está inicializado en el proceso root.
 - N y P son potencias de 2, siendo $N \gg P$.
 - En las operaciones colectivas MPI los *buffers* de envío y recepción pueden solaparse.
 - El resultado debe ser correcto para todos los procesos.
- (a) [0.5p] Diseña la paralelización del Algoritmo A. Por ejemplo, escribe un pseudocódigo de alto nivel indicando los pasos que se deben seguir. Puedes utilizar también una descripción gráfica.
- (b) [1.5p] Implementa la función `stdev_mpi` como se describe, siguiendo tu diseño propuesto. Utiliza funciones colectivas de MPI siempre que sea posible.
- (c) [0.5p] Piensa en cómo se paralelizaría la función del Algoritmo B. ¿Cuál de las dos implementaciones sería más eficiente?

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
```

SOLUCIÓN

1. (a) Es una descomposición de dominio, siendo el dominio el conjunto de maquetas.

La asignación de tareas durante el procesamiento es irregular (ni por bloques, ni cíclica) pero estática, dado que al leer las maquetas podemos determinar exactamente su tiempo de computación, con lo que podemos hacer un reparto equilibrado a priori.

También se puede mencionar una descomposición funcional sobre el problema completo (lectura de maquetas, evaluación paralela y postprocesado secuencial).

- (b) Secuencialmente tenemos que

- leer las 64 maquetas: $64 \times 0.1 \text{ s} = 6.4 \text{ s}$
- procesarlas. El tiempo mínimo sería si todas fuesen de 3 minutos, y el máximo si todas fuesen de 6 minutos. Esta es la única parte paralelizable del algoritmo, con lo que de acuerdo a la ley de Amdahl, la mayor aceleración posible se obtendrá cuando esta parte el algoritmo ocupe la mayor porción de tiempo posible, lo cual sucedería si todas las canciones fuesen de 6 minutos:
 $64 \text{ canciones} \times (6 \text{ minutos} \times 60 \text{ s./minuto} \times 8 \text{ s.proc/s. sonido}) = 64 \times 2880 = 184320 \text{ s}$
- hacer el postprocesado con ordenamiento: $64 \times \log_2(64) = 64 \times 6 = 384 \text{ s}$

Totalizando $T_{seq} = 6.4 + 184320 + 384 = 184710.4 \text{ s}$

En paralelo se procesarían las 64 canciones, quedando un tiempo $T_{par} = 6.4 + 2880 + 384 = 3270.4 \text{ s}$ y siendo la aceleración asociada $A = 184710.4/3270.4 = 56.48$

- (c) El mínimo número de servidores serían 64, con lo que

- $Ef = 56.48/64 = 0.88 = 88\%$
- $Coste = 3270.4 \text{ s} \times 64 = 209305.6 \text{ s}$
- $Sobrecarga = Coste - T_{seq} = 209305.6 - 184710.4 = 24595.2 \text{ s}$

- (d) Los tiempos de análisis de las maquetas dejarían de ser predecibles a priori, con lo que un reparto estático dejaría de garantizar una carga equilibrada entre los procesadores. Si bien la descomposición seguiría siendo de dominio, la asignación de tareas pasaría a ser dinámica. Dado que sólo un servidor tiene acceso al disco de las maquetas, lo más fácil sería implementar un esquema centralizado maestro/esclavo, pero dado que el tiempo de transmisión de las maquetas es ínfimo y nada parece impedir que todos los servidores se comuniquen, se podría hacer una distribución inicial por bloques o cíclica, y luego aplicar un esquema de asignación dinámico descentralizado.

2. (a) El algoritmo tendrá los pasos siguientes:

- 1 Repartir vector v
- 2 Cálculo paralelizado de $sum = \sum(v_i)$ (con reducción de tipo suma)
- 3 Distribución de sum a todos los procesos (broadcast)
- 4 Todos los procesos calculan la media (μ)
- 5 Cálculo paralelizado de $sum = \sum(v_i - \mu)^2$ (con reducción de tipo suma)
- 6 Distribución de sum a todos los procesos (broadcast)
- 7 Todos los procesos calculan $sd = \sqrt{sum/N}$

También es válido que sólo el proceso root calcule μ y sd y distribuya estas variables en lugar de distribuir sum . En ese caso se invertiría el orden de los pasos [3,4], y [6,7].

- (b) Una posible implementación es la siguiente:

```

double stdev_mpi(double *v, int N, int R, int P, int root) {

    double v_local[N/P];

    MPI_Scatter(v,          N/P, MPI_DOUBLE,
               v_local, N/P, MPI_DOUBLE, root, MPI_COMM_WORLD);

    sum = 0;
    for (i=0; i < N/P; i++)
        sum += v_local[i]

    MPI_Reduce(&sum, &sum, 1, MPI_DOUBLE, MPI_SUM, root, MPI_COMM_WORLD);
    MPI_Bcast( &sum, 1, MPI_DOUBLE, root, MPI_COMM_WORLD);

    media = sum / N;

    sum = 0;
    for (i=0; i < N/P; i++)
        sum += pow(v_local[i] - media,2);

    MPI_Reduce(&sum, &sum, 1, MPI_SUM, MPI_DOUBLE, root, MPI_COMM_WORLD);
    MPI_Bcast( &sum, 1, MPI_DOUBLE, root, MPI_COMM_WORLD);

    sd = sqrt(sum / n);

    return sd;
}

```

- (c) En el Algoritmo B no es necesaria la separación de tareas entre el cálculo de la media y la desviación estándar. En una única tarea se calcula la suma y la suma de cuadrados, y para finalizar se realizan dos operaciones que pueden ejecutar todos los procesos sin un efecto significativo en el rendimiento.

Esta implementación sería más eficiente porque no requiere la sincronización necesaria entre las dos tareas del algoritmo A.